

Lecture 03: Methods

Introduction to Programming with Java

Today's learning goals

By the end of this lecture you should be able to:

1. Explain why we use methods to structure programs
2. Declare a method with parameters and a return type
3. Call methods and use returned values
4. Recognise simple method overloading
5. Trace a short program that uses several methods

Agenda

1. Why methods?
2. Anatomy of a method
3. Parameters and arguments
4. Return values and `void`
5. Scope (local variables)
6. Overloading (same name, different parameter lists)
7. Putting it together

The problem with “one long main”

- Programs grow: copying and pasting logic becomes messy
- Hard to test one idea in isolation
- Hard to read: what does this block *mean*?

Idea

A **method** is a named recipe: inputs in, optional result out.

You already used methods

- `System.out.println(...)` — prints a line
- `s.length()`, `s.charAt(i)`, `s.equals(t)`
- `main` itself is a method (the program entry point)

Today you learn to **write** your own.

Benefits

- **Reuse:** write once, call many times
- **Clarity:** name the intention (`average`, `isEven`)
- **Testing:** check a small piece with known inputs
- **Division of labour:** later, methods live in classes (next lectures)

A first method

```
1 public class Demo {  
2     public static void main(String[] args) {  
3         greet("Ada");  
4         greet("Alan");  
5     }  
6  
7     public static void greet(String name) {  
8         System.out.println("Hello, " + name + "!");  
9     }  
10 }
```

Parts of a method header

For now we keep methods `public static` (like `main`) so they can be called from `main` without objects—objects come next lecture.

- **Return type:** `void` or e.g. `int`, `double`, `boolean`
- **Name:** `camelCase` (`computeAverage`)
- **Parameter list:** types + names in parentheses
- **Body:** statements between braces

Calling a method

```
1 greet("Ada");           // argument "Ada" fills parameter name
2
3 int x = doubleIt(7);
4 System.out.println(x);   // 14
5
6 public static int doubleIt(int n) {
7     return n * 2;
8 }
```

Argument = value you pass; **parameter** = variable that receives it.

Quiz 1 — Call vs definition

```
1 public static void shout(String msg) {  
2     System.out.println(msg + "!");  
3 }
```

Quiz

Which line correctly *calls* this method?

1. `public static void shout(String msg)`
2. `shout("Go")`
3. `void shout("Go")`
4. `return shout;`

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 1

Answer

2. `shout("Go")` — use the name and pass arguments.

Multiple parameters

```
1 public static int add(int a, int b) {  
2     return a + b;  
3 }  
4  
5 // in main:  
6 int sum = add(3, 5);           // 8  
7 int sum2 = add(10, -2);       // 8
```

Order and types of arguments must match the parameter list.

Return sends a value back

```
1 public static boolean isEven(int n) {  
2     return n % 2 == 0;  
3 }  
4  
5 if (isEven(10)) {  
6     System.out.println("even");  
7 }
```

After return, the method finishes—later lines in that method do not run.

void means “no value returned”

```
1 public static void printStars(int n) {  
2     for (int i = 0; i < n; i++) {  
3         System.out.print("*");  
4     }  
5     System.out.println();  
6 }
```

Common mistake

Writing `return 5;` inside a void method, or forgetting `return` in a non-void method.

Pass-by-value (student view)

For primitive arguments (`int`, `double`, ...):

- The method receives a **copy** of the value
- Changing the parameter does not change the caller's variable

(Arrays/objects are references—we refine that when we discuss objects.)

Demo: copy of an int

```
1 public static void tryChange(int n) {  
2     n = 100;  
3 }  
4  
5 public static void main(String[] args) {  
6     int x = 5;  
7     tryChange(x);  
8     System.out.println(x);    // still 5  
9 }
```

Quiz 2 — What is returned?

```
1 public static int mystery(int a, int b) {  
2     if (a > b) {  
3         return a;  
4     }  
5     return b;  
6 }
```

Quiz

What does `mystery(3, 8)` return?

1. 3
2. 8
3. 11
4. Nothing (void)

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. 8 — the method returns the larger of the two values.

Local variables

```
1 public static double average(int[] xs) {  
2     int sum = 0;                                // local to this method  
3     for (int i = 0; i < xs.length; i++) {  
4         sum += xs[i];  
5     }  
6     return (double) sum / xs.length;  
7 }
```

sum and i exist only inside the method (and loop). Callers cannot see them.

Scope rules of thumb

- Declare variables in the smallest useful region
- Parameter names are local to the method
- Two methods may reuse the same local name independently

Same name, different parameters

Overloading: several methods share a name if their **parameter lists** differ (number or types of parameters). The return type alone is *not* enough to distinguish them.

Overloading example

```
1 public static int max(int a, int b) {  
2     return (a > b) ? a : b;  
3 }  
4  
5 public static double max(double a, double b) {  
6     return (a > b) ? a : b;  
7 }  
8  
9 public static int max(int a, int b, int c) {  
10     return max(max(a, b), c);  
11 }
```

Why overload?

- One idea (“maximum”) for related types / arities
- Callers write `max(...)` naturally
- Compiler picks the matching version

Try this

Add `max` for two `long` values, or for an `int[]` (find max in an array).

Quiz 3 — Overloading (true/false)

Quiz

True or false?

You can overload two methods that have the same parameter types and differ only in return type (e.g. both take one `int`, one returns `int`, one returns `double`).

1. True
2. False

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

False. Parameter lists must differ; return type alone is not enough.

Mini program: grade helper

```
1 public static double average(int[] scores) {  
2     int sum = 0;  
3     for (int s : scores) {  
4         sum += s;  
5     }  
6     return (double) sum / scores.length;  
7 }  
8  
9 public static String label(double avg) {  
10     if (avg >= 90) return "Excellent";  
11     if (avg >= 70) return "Good";  
12     return "Keep practising";  
13 }
```

Calling from main

```
1 public static void main(String[] args) {  
2     int[] scores = {80, 90, 70, 100};  
3     double avg = average(scores);  
4     System.out.println("Average: " + avg);  
5     System.out.println(label(avg));  
6 }
```

- One clear job per method
- Prefer returning a value over only printing (easier to reuse)
- Keep `main` short: set up data, call helpers, show results

Key takeaways

1. Methods name reusable pieces of behaviour
2. Parameters receive arguments; `return` sends a result (unless `void`)
3. Local variables have method (or block) scope
4. Overloading: same name, different parameter lists

Practice before next time

Try this

Write `countVowels(String s)` and `max(int[] values)`. Call both from `main`.

Coming next

Classes and objects: fields, constructors, and creating instances with `new`

Questions?

Parameters, return, or overloading—what should we trace once more?